Lab Assignment-4

School of Engineering and Technology
Course Name & Code: Fundamentals of Operating System Lab (ENCA252)
Program Name: BCA (AI & DS) (Research)

Submission Guidelines

General Requirements

- Submission Deadline: Assignments must be submitted within one week of the assignment's release date.
- Submission Platform: All assignments are to be submitted via LMS.
- GitHub Link: Provide a GitHub repository link with your submission.
- Individual Submission: Each student must submit individually. Formatting: Follow the prescribed format strictly.

Evaluation

- **Total Marks:** This assignment is worth a total of 5 marks.
- **Evaluation Metrics:** Assignments will be evaluated based on the following criteria:
  - **Originality:** The uniqueness and independent thought demonstrated in the work.
  - **Correctness:** The accuracy and validity of the solutions or content.

Problem Title:

Implementation and Analysis of Disk Scheduling Algorithms

Problem Statement

Disk scheduling is an essential function of operating systems that determines the order in which disk I/O requests are serviced. Efficient disk scheduling improves system performance by reducing seek time and increasing throughput.

In this assignment, students will implement and analyze various disk scheduling algorithms such as **FCFS, SSTF, SCAN, and C-SCAN**. The goal is to simulate head movements and calculate total seek time.

Learning Objectives

- Understand disk scheduling concepts
- Implement various disk scheduling algorithms
- Calculate total head movement (seek time)
- Compare performance of different algorithms

Tools/Technology Used

- Python 3.x
- Basic Python libraries
- Any IDE (VS Code / PyCharm / IDLE)
- Linux/ Ubuntu

Assignment Tasks

Task 1: Input and Disk Request Representation

Objective:

To understand how disk requests are handled.

Description:

Students will input:

- Disk request queue
- Initial head position
- Disk size (optional) **Sub-Tasks:**

1. Input number of disk requests
2. Input request sequence
3. Input initial head position

Expected Outcome:

- Understanding of disk queue structure
- Ability to simulate disk head movement

Task 2: FCFS Disk Scheduling

Objective:

To implement First Come First Serve (FCFS).

Description:

Requests are serviced in the order they arrive.

Sub-Tasks:

1. Traverse requests sequentially
2. Calculate total seek time
3. Display movement sequence **Expected Outcome:**

- Understanding simple scheduling
- Calculation of seek time

Sample Python Logic (FCFS)

```
def fcfs(requests, head):
    seek_time = 0
for req in requests:
        seek_time += abs(head - req)
head = req
                                        print("Total Seek Time:", seek_time)
```

Task 3: SSTF (Shortest Seek Time First)

Objective:

To implement SSTF algorithm.

Description:

Selects the request closest to current head position.

Sub-Tasks:

1. Find nearest request
2. Update head position
3. Calculate seek time

Expected Outcome:

- Understanding optimization strategy
- Reduced seek time compared to FCFS

Sample Python Logic (SSTF)

```
def sstf(requests, head):
seek_time = 0
    requests = requests.copy()
     while requests:         nearest = min(requests,
key=lambda x: abs(x - head))         seek_time += abs(head -
nearest)         head = nearest
requests.remove(nearest)

                                        print("Total Seek Time:", seek_time)
```

Task 4: SCAN (Elevator Algorithm)

Objective:

To implement SCAN algorithm.

Description:

Disk head moves in one direction servicing requests, then reverses direction.

Sub-Tasks:

1. Sort request queue
2. Move in one direction
3. Reverse direction at end

Expected Outcome:

- Understanding elevator movement
- Balanced performance

Sample Python Logic (SCAN)

```
def scan(requests, head, disk_size):
seek_time = 0
    left = [r for r in requests if r < head]
right = [r for r in requests if r >= head]

    left.sort(reverse=True)
right.sort()
```

```
     for r in right:
seek_time += abs(head - r)
head = r

    seek_time += abs(head - (disk_size - 1))
head = disk_size - 1
     for r in left:
seek_time += abs(head - r)
head = r

                                        print("Total Seek Time:", seek_time)
```

Task 5: C-SCAN Algorithm

Objective:

To implement Circular SCAN.

Description:

Head moves in one direction only and jumps back to the beginning.

Sub-Tasks:

1. Move in one direction
2. Jump to start
3. Continue servicing **Expected Outcome:**

- Uniform wait time
- Improved fairness

Sample Python Logic (C-SCAN)

```
def cscan(requests, head, disk_size):
seek_time = 0
    left = [r for r in requests if r < head]
right = [r for r in requests if r >= head]
     left.sort()
right.sort()
for r in right:
seek_time +=
abs(head - r)
head = r

    seek_time += abs(head - (disk_size - 1))
head = 0
    seek_time += disk_size - 1
```

```
    for r in left:
seek_time += abs(head - r)
head = r

print("Total Seek Time:", seek_time)
```

Task 6: Performance Comparison

Objective:

To compare all algorithms.

Description:

Students will compare:

- Total seek time
- Efficiency **Sub-Tasks:**

1. Run all algorithms
2. Compare outputs
3. Identify best algorithm

Expected Outcome:

- Analytical understanding
- Performance comparison

Task 7: Result Analysis

Objective:

To interpret results.

Description:

Students will:

- Analyze which algorithm performs best
- Justify results **Sub-Tasks:**

1. Compare seek times
2. Identify pros/cons
3. Write conclusion **Expected Outcome:**

- Concept clarity

- Real-world understanding

Code –

```
# FCFS (First Come First Serve)

def fcfs(requests, head):

    seek_time = 0

    sequence = [head]

    for req in requests:

        seek_time += abs(head - req)

        head = req

        sequence.append(head)

    return seek_time, sequence

# SSTF (Shortest Seek Time First)

def sstf(requests, head):

    seek_time = 0

    requests = requests.copy()

    sequence = [head]

    while requests:

        nearest = min(requests, key=lambda x: abs(x - head))

        seek_time += abs(head - nearest)

        head = nearest

        sequence.append(head)

        requests.remove(nearest)

    return seek_time, sequence

# SCAN (Elevator Algorithm)
```

```
def scan(requests, head, disk_size, direction="right"):
    seek_time = 0
    sequence = [head]
    left = [r for r in requests if r < head]
    right = [r for r in requests if r >= head]
    left.sort(reverse=True)
    right.sort()
    if direction == "right":
        for r in right:
            seek_time += abs(head - r)
            head = r
            sequence.append(head)
        # go to end
        seek_time += abs(head - (disk_size - 1))
        head = disk_size - 1
        sequence.append(head)
        for r in left:
            seek_time += abs(head - r)
            head = r
            sequence.append(head)
    else:  # left direction
        for r in left:
            seek_time += abs(head - r)
            head = r
```

```
            sequence.append(head)
        seek_time += abs(head - 0)
        head = 0
        sequence.append(head)
        for r in right:
            seek_time += abs(head - r)
            head = r
            sequence.append(head)
    return seek_time, sequence=
# C-SCAN (Circular SCAN)
def cscan(requests, head, disk_size):
    seek_time = 0
    sequence = [head]
    left = [r for r in requests if r < head]
    right = [r for r in requests if r >= head]
    left.sort()
    right.sort()
    # service right side
    for r in right:
        seek_time += abs(head - r)
        head = r
        sequence.append(head)
    # go to end
    seek_time += abs(head - (disk_size - 1))
```

```
    head = disk_size - 1
    sequence.append(head)
    # jump to start
    seek_time += disk_size - 1
    head = 0
    sequence.append(head)
    # service left side
    for r in left:
        seek_time += abs(head - r)
        head = r
        sequence.append(head)
    return seek_time, sequence
# Main Program
def main():
    print("Disk Scheduling Algorithms Simulation\n")
    n = int(input("Enter number of requests: "))
    requests = list(map(int, input("Enter request sequence: ").split()))
    head = int(input("Enter initial head position: "))
    disk_size = int(input("Enter disk size: "))
    print("\nRequest Queue:", requests)
    print("Initial Head Position:", head)


    fcfs_time, fcfs_seq = fcfs(requests, head)
    sstf_time, sstf_seq = sstf(requests, head)
```

```
    scan_time, scan_seq = scan(requests, head, disk_size)
    cscan_time, cscan_seq = cscan(requests, head, disk_size)
    print("\n--- Results ---")
    print("FCFS Seek Time:", fcfs_time)
    print("Sequence:", fcfs_seq)
    print("\nSSTF Seek Time:", sstf_time)
    print("Sequence:", sstf_seq)
    print("\nSCAN Seek Time:", scan_time)
    print("Sequence:", scan_seq)
    print("\nC-SCAN Seek Time:", cscan_time)
    print("Sequence:", cscan_seq)
    results = {
        "FCFS": fcfs_time,
        "SSTF": sstf_time,
        "SCAN": scan_time,
        "C-SCAN": cscan_time
    }
    best = min(results, key=results.get)
    worst = max(results, key=results.get)
    print("\n--- Analysis ---")
    print(f"Best Algorithm: {best} (Minimum Seek Time = {results[best]})")
    print(f"Worst Algorithm: {worst} (Maximum Seek Time = {results[worst]})")
if __name__ == "__main__":
    main()
```

Output-

```
harinand@Lenovo:~/Desktop$ nano OsAss4.py
harinand@Lenovo:~/Desktop$ nano OsAss4.py
harinand@Lenovo:~/Desktop$ python3 OsAss4.py
Disk Scheduling Algorithms Simulation

Enter number of requests: 8
Enter request sequence: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter disk size: 200

Request Queue: [98, 183, 37, 122, 14, 124, 65, 67]
Initial Head Position: 53

--- Results ---
FCFS Seek Time: 640
Sequence: [53, 98, 183, 37, 122, 14, 124, 65, 67]

SSTF Seek Time: 236
Sequence: [53, 65, 67, 37, 14, 98, 122, 124, 183]

SCAN Seek Time: 331
Sequence: [53, 65, 67, 98, 122, 124, 183, 199, 37, 14]

C-SCAN Seek Time: 382
Sequence: [53, 65, 67, 98, 122, 124, 183, 199, 0, 14, 37]

--- Analysis ---
Best Algorithm: SSTF (Minimum Seek Time = 236)
Worst Algorithm: FCFS (Maximum Seek Time = 640)
harinand@Lenovo:~/Desktop$
```

Student Instructions

- **Submit Python code and output screenshots in Linux/Ubuntu**
- Maintain proper comments and structure
- Submit report (.pdf/.docx) with explanation
- Late submissions will be penalized

Rubrics for Evaluation

Criterion	Weight	Excellent (5)	Good (4)	Fair/Poor (≤3)

Implementation & Execution	40%	All algorithms correctly implemented	Minor issues	Incorrect/incomplete
Code Quality & Documentation	30%	Well-structured code	Moderate clarity	Poor formatting
Output Accuracy	20%	Correct results	Minor mismatch	Incorrect
Report & Submission	10%	On time & clear	Minor delay	Late/no submission